perplexity

okay, what do we have so far

You now have a complete outline for a role-based moderation system layered onto your existing Discord bot. It separates **staff operations**, **high-privilege administration**, **warning automation**, and **auditable configuration**.

Dashboard structure

```
General
- Overview
- Analytics

Tickets
- Configurations
- Trial Schedules
- Transcripts

Staffing
- Warnings
- Expired Warnings
- Blacklist
- Staffing Viewing

Admin Panel
- Access Control
- Warning Builder
- Warning Log Settings
- Audit Logs
- Admin Panel Access
```

Staffing

Page	Purpose	Intended access
Warnings	Active warning records: target, level, reason, moderator, timeout result, expiry, matching role, and DM state	Staffing Viewing roles
Expired Warnings	Read-only warning history after a record expires	Staffing Viewing roles
Blacklist	Display and manage ticket blacklists, including Normal/Voidcore classification	Blacklist Management roles
Staffing Viewing	Controls which roles can see Staffing pages	Admin Panel users

Admin Panel

Page	Purpose
Access Control	Manage dashboard, ticket, staffing, warning, blacklist, and command permissions
Warning Builder	Design warning-related messages and responses
Warning Log Settings	Select official log channel; enable/disable logging; choose log format/template
Audit Logs	Review significant moderation and configuration events
Admin Panel Access	Grant/revoke access to Admin Panel

The Admin Panel is hidden and protected for ordinary dashboard users. Only Render-configured superadmins or people explicitly granted Admin Panel access can enter it; only environment-configured superadmins should be able to modify the Admin Panel access list. The codebase already has an environment-based administrator concept in its access helpers.[1]

Warning slash commands

Each warning command has exactly two required options:

```
/w1 users:<member> reason:<reason>
/w2 users:<member> reason:<reason>
/w3 users:<member> reason:<reason>
```

`users` is a single selected Discord member despite the plural option name. No bulk warning behavior is planned.

Command	Required options	Role applied	Native timeout	Warning expires after
`/w1`	`users`, `reason`	W1	6 hours	14 days
`/w2`	`users`, `reason`	W2	12 hours	21 days
`/w3`	`users`, `reason`	W3	1 day	28 days

Every command will:

1. Require a nonblank reason.
2. Check Warning Management authorization.
3. Check that the bot can manage the member and matching warning role.
4. Add the matching W1/W2/W3 role.
5. Apply Discord's native timeout.
6. Save a persistent moderation record.
7. Attempt to send a warning DM.
8. Send an official warning-log entry.
9. Send an audit-log record.

10. Return an ephemeral staff result.

At the warning expiry date, the bot moves the record from active to expired history and removes the associated warning role. This should be done by a periodic, restart-safe expiry worker—not by a high-frequency loop.

Warning Builder

ADMIN PANEL → Warning Builder controls five separately editable templates:

Template	Trigger	Recipient
Success confirmation	Warning, role, timeout, and DM succeed	Issuing moderator, ephemeral
Success — DM unavailable	Moderation action succeeds, but the member cannot be DMed	Issuing moderator, ephemeral
Failure response	Warning action fails before completion	Issuing moderator, ephemeral
Warning DM	Warning action succeeds	Warned member
Warning Log	Warning action succeeds	Configured log channel

Each template offers:

```
Plain Message | Embed | Components V2
```

Useful placeholders include:

```
{member_mention}
{member_name}
{member_id}
{moderator_mention}
{moderator_name}
{moderator_id}
{warning_level}
{reason}
{timeout_duration}
{timeout_ends_at}
{warning_duration}
{warning_expires_at}
{warning_id}
{guild_name}
```

When DM delivery fails, the bot uses the dedicated editable **Success — DM unavailable** response. The accurate default wording is:

> The member could not be DMed. The warning was still issued successfully.

Components V2 must be handled as its own renderer because Discord's `IS_COMPONENTS_V2` flag prevents mixing it with ordinary top-level content and embeds in the same message.[2] [3]

Warning logs and audits

ADMIN PANEL → Warning Log Settings controls:

- Whether warning logs are enabled
- The destination moderation-log channel
- The warning-log format: message, embed, or Components V2
- The warning-log template and which details it includes

A log can include the member, moderator, warning level, reason, timeout duration/end, warning expiry, warning ID, and DM-delivery state.

ADMIN PANEL → Audit Logs records:

- Warning issued, failed, expired, or manually removed
- Role application/removal and timeout outcomes
- DM delivery/unavailable state
- Blacklist add/remove/expiry actions and Normal/Voidcore type
- Changes to warning templates and warning-log settings
- Access Control and Staffing Viewing changes
- Admin Panel-access changes

Permissions and intents

For warnings and role assignment, the bot needs:

- **Manage Roles** to add and later remove W1/W2/W3 roles
- **Moderate Members** to apply native timeouts
- Bot role positioned above warning roles and target members
- Appropriate channel permissions to send logs

You do **not** need Server Members Intent specifically for this slash-command workflow. Discord provides the member selected in the interaction, and the bot can fetch only that one person if necessary. You also do not need Message Content or Presence Intent for required slash-command reasons, warnings, timeouts, and direct selected-user role assignment. Your current configuration has both member and message-content intents off, which can remain that way for this design. [4]

Storage and rate limiting

Use MongoDB via `MONGODB_URI` in Render for warnings, audit events, warning-builder settings, permissions, and blacklists. The current project supports MongoDB with local JSON fallback, but local runtime files are not reliable production storage on a typical Render deployment. [5] [6]

This plan should not cause global rate-limit issues: one warning usually performs only a few controlled Discord operations—role update, timeout, optional DM, log message, and interaction

response. Keep actions single-user, sequential, and bounded; do not add mass-warning commands or immediate retry loops. Discord documents temporary restrictions for IPs that generate very high volumes of invalid requests, including repeated ignored error responses.[7]

Remaining decisions

- **Warning-role strategy:** cumulative (W1 + W2 + W3) versus highest-active-only.
- Whether authorized staff can manually remove/reduce a warning before expiry.
- The exact roles/user IDs for Warning Management, Blacklist Management, Staffing Viewing, Admin Panel, and superadmin access.
- Whether Voidcore blacklists only block tickets or create extra restrictions.
- The exact default designs/wording for each Warning Builder template.
- Whether Admin Panel users can edit all Access Control settings or only Render-defined superadmins can.

⁂

1. access.py
2. https://docs.discord.com/developers/components/reference
3. https://docs.discord.com/developers/components/overview
4. main.py
5. db.py
6. storage.py
7. https://github.com/discord/discord-api-docs/blob/main/developers/topics/rate-limits.mdx